In [367...
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [368...
```python
df=pd.read_csv("data_poly.csv")
```

In [369...
```python
df.head()
```

Out[369...

	Experience_Years	Package_LPA
0	1	27.8
1	2	15.2
2	3	18.2
3	4	48.8
4	5	43.0

In [370...
```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 2 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   Experience_Years  100 non-null    int64  
 1   Package_LPA       100 non-null    float64
dtypes: float64(1), int64(1)
memory usage: 1.7 KB
```

In [371...
```python
df.describe()
```

Out[371...

	Experience_Years	Package_LPA
count	100.000000	100.000000
mean	50.500000	2826.210000
std	29.011492	2476.286232
min	1.000000	15.200000
25%	25.750000	599.850000
50%	50.500000	2163.400000
75%	75.250000	4713.450000
max	100.000000	8216.000000

In [372...
```python
df.isnull()
```

Out[372...

	Experience_Years	Package_LPA
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
95	False	False
96	False	False
97	False	False
98	False	False
99	False	False

100 rows × 2 columns

In [373...

```
df.isnull().sum()
```

Out[373...

```
Experience_Years    0
Package_LPA         0
dtype: int64
```

In [374...

```
df.duplicated().sum()
```

Out[374...

```
np.int64(0)
```

In [375...

```
val_num=df.select_dtypes(include=['int64','float64']).columns
val_num
```

Out[375...

```
Index(['Experience_Years', 'Package_LPA'], dtype='object')
```

In [376...

```
q1=df[val_num].quantile(0.25)
q3=df[val_num].quantile(0.75)
IQR=q3-q1

q1,q3,IQR
```

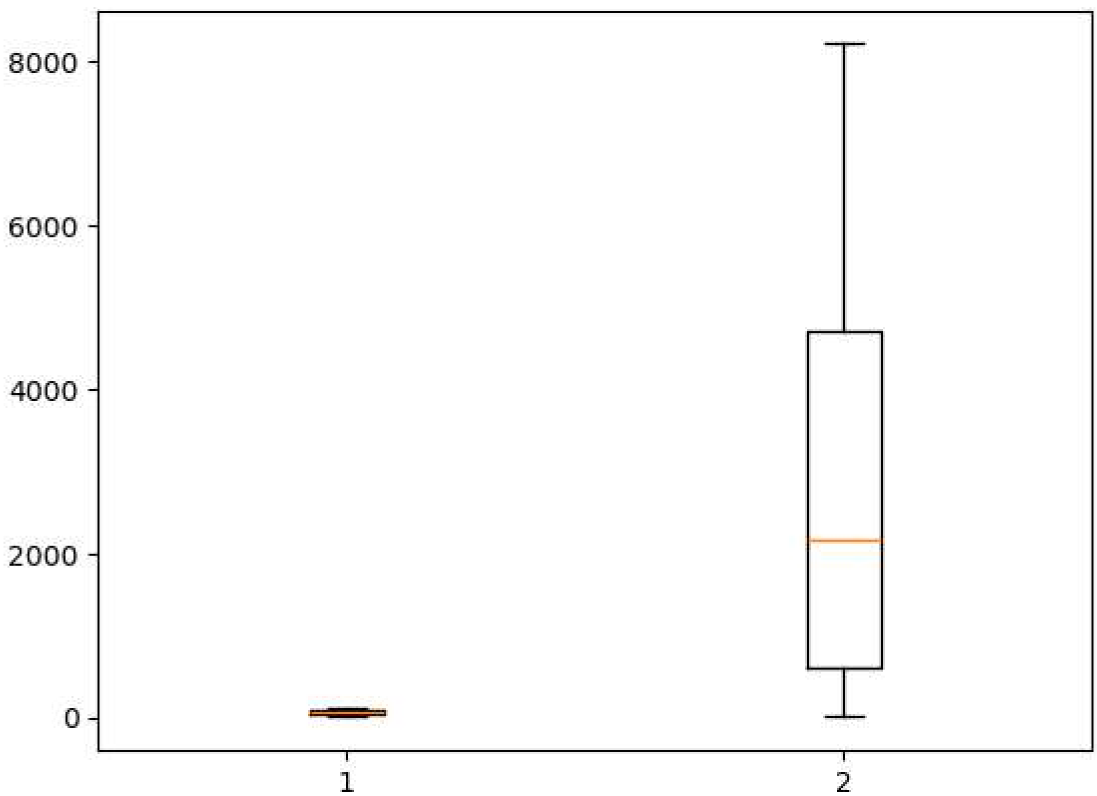
Out[376...

```
(Experience_Years     25.75
 Package_LPA         599.85
 Name: 0.25, dtype: float64,
 Experience_Years      75.25
 Package_LPA         4713.45
 Name: 0.75, dtype: float64,
 Experience_Years      49.5
 Package_LPA         4113.6
 dtype: float64)
```

In [377…
```
upper_limit=q3+(1.5*IQR)
lower_limit=q1-(1.5*IQR)

upper_limit,lower_limit
```

Out[377…
```
(Experience_Years      149.50
 Package_LPA         10883.85
 dtype: float64,
 Experience_Years     -48.50
 Package_LPA        -5570.55
 dtype: float64)
```

In [378…
```
plt.boxplot(df[val_num])
plt.show()
```



In [379…
```
q1=df['Package_LPA'].quantile(0.25)
q3=df['Package_LPA'].quantile(0.75)
IQR=q3-q1

q1,q3,IQR
```

Out[379…
```
(np.float64(599.8499999999999), np.float64(4713.45), np.float64(4113.6))
```

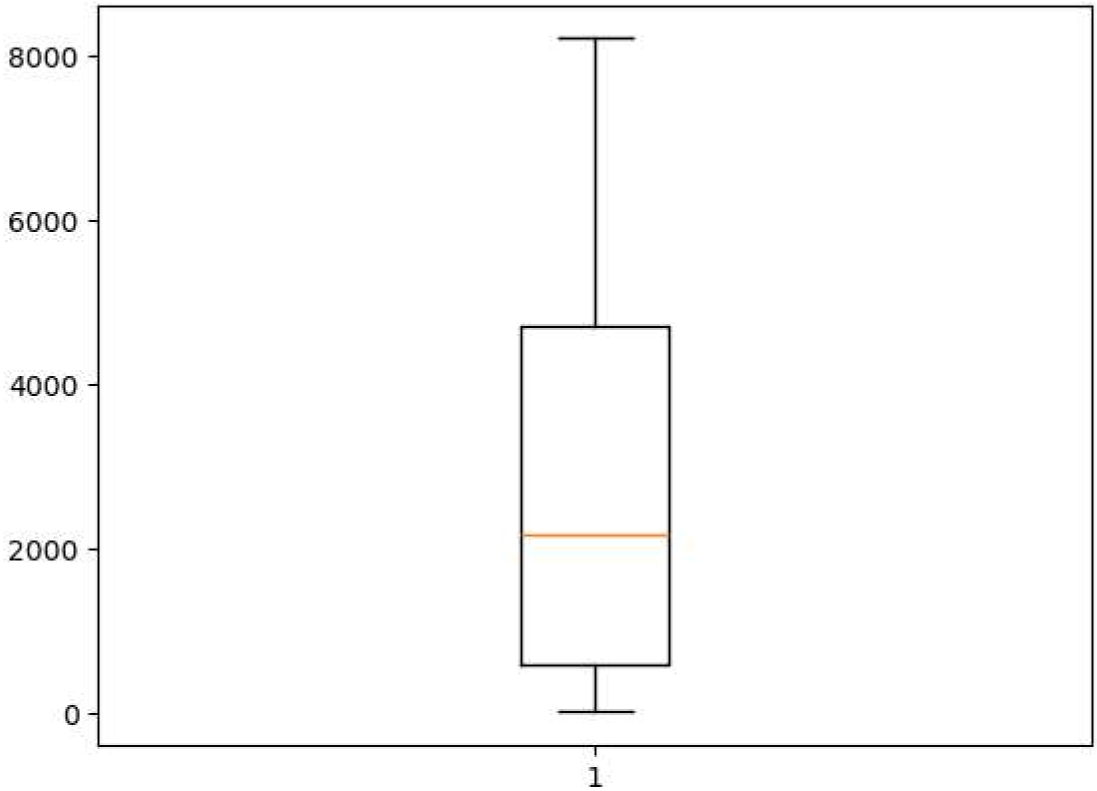
In [380…
```
upper_limit=q3+(1.5*IQR)
lower_limit=q1-(1.5*IQR)

upper_limit,lower_limit
```

Out[380…
```
(np.float64(10883.85), np.float64(-5570.550000000001))
```

In [381…
```
df=df[(df['Package_LPA'] >= lower_limit)&(df['Package_LPA']<=upper_limit)]
```

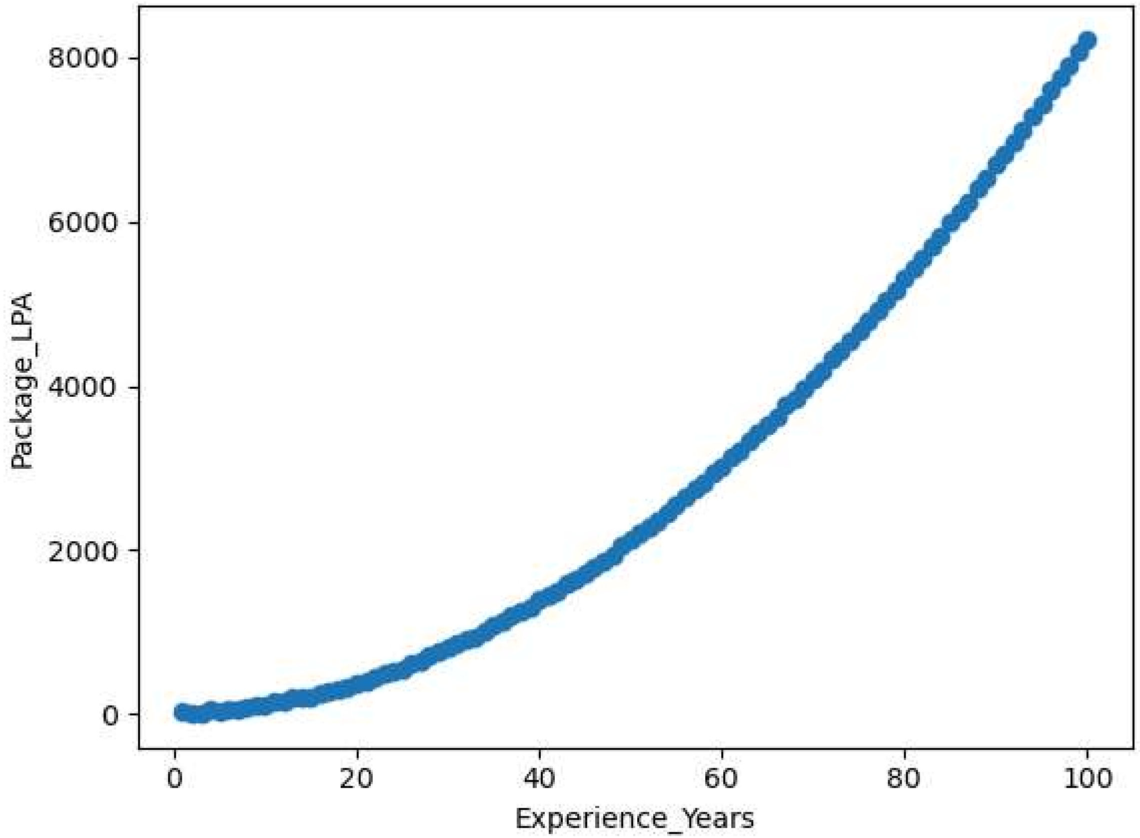
In [382…
```
df.reset_index(drop=True, inplace=True)
```

In [383…
```
plt.boxplot(df['Package_LPA'])
plt.show()
```



In [384…
```
plt.scatter(df['Experience_Years'],df['Package_LPA'])
plt.xlabel('Experience_Years')
plt.ylabel('Package_LPA')
```

Out[384…
```
Text(0, 0.5, 'Package_LPA')
```



In [385…

```
x=df[['Experience_Years']]
y=df[['Package_LPA']]

x,y
```

Out[385…
```
(    Experience_Years
 0                  1
 1                  2
 2                  3
 3                  4
 4                  5
 ..               ...
 95                96
 96                97
 97                98
 98                99
 99               100

 [100 rows x 1 columns],
     Package_LPA
 0          27.8
 1          15.2
 2          18.2
 3          48.8
 4          43.0
 ..          ...
 95       7595.8
 96       7737.2
 97       7889.2
 98       8054.8
 99       8216.0

 [100 rows x 1 columns])
```

In [386…
```
from sklearn.model_selection import train_test_split
```

In [387…
```
x_train,x_test,y_train,y_test=train_test_split(x,y,random_state=42,test_size=0.2)
```

In [388…
```
print(x_train.shape)
print(x_test.shape)
```

```
(80, 1)
(20, 1)
```

In [389…
```
from sklearn.linear_model import LinearRegression
```

In [390…
```
reg=LinearRegression()
reg.fit(x_train,y_train)
LinearRegression()
```

Out[390…

▾ LinearRegression

▶ Parameters

In [391…
```
y_pred=reg.predict(x_test)
```

In [392…
```
from sklearn.preprocessing import PolynomialFeatures
```

In [393...
```
poly_reg=PolynomialFeatures(degree=3)
```

In [394...
```
x_poly=poly_reg.fit_transform(x)
poly_reg.fit(x_poly,y)
```

Out[394...

▾ PolynomialFeatures

▶ Parameters

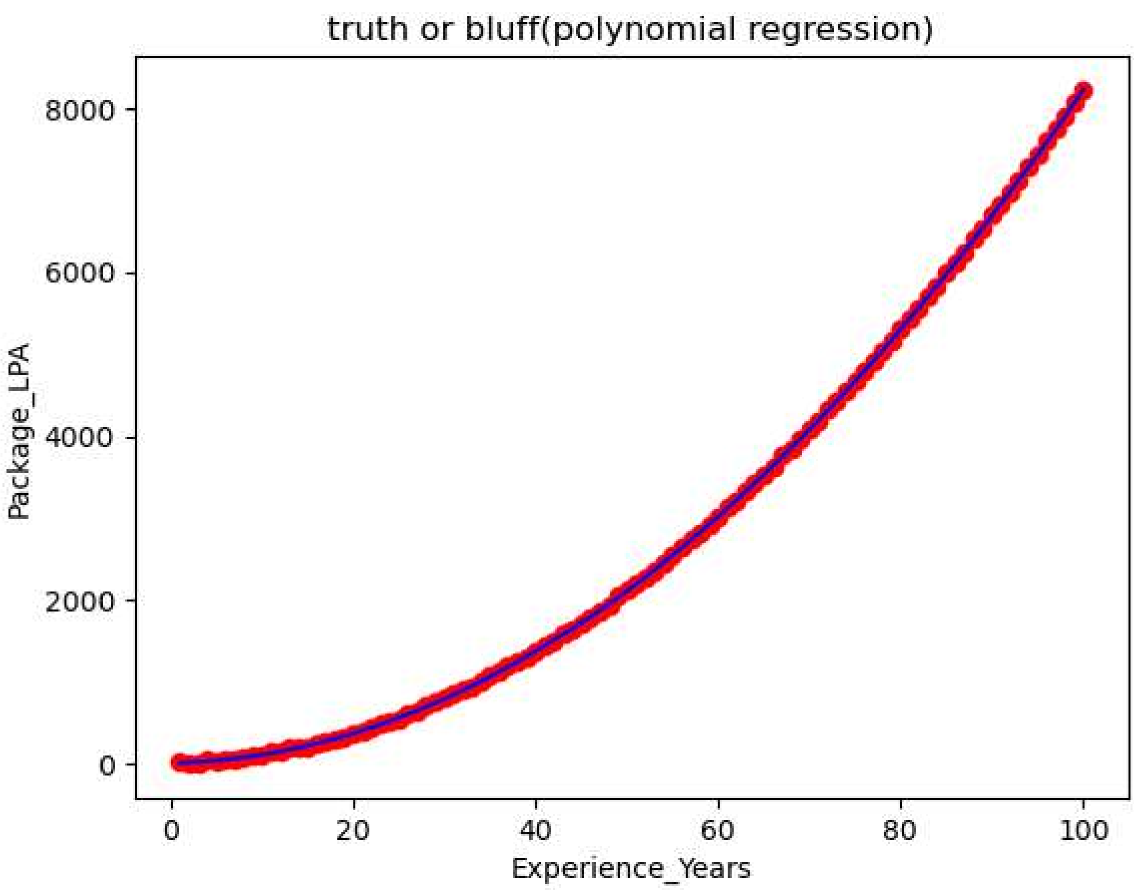
In [395...
```
lin_reg_2=LinearRegression()
lin_reg_2.fit(x_poly,y)
```

Out[395...

▾ LinearRegression

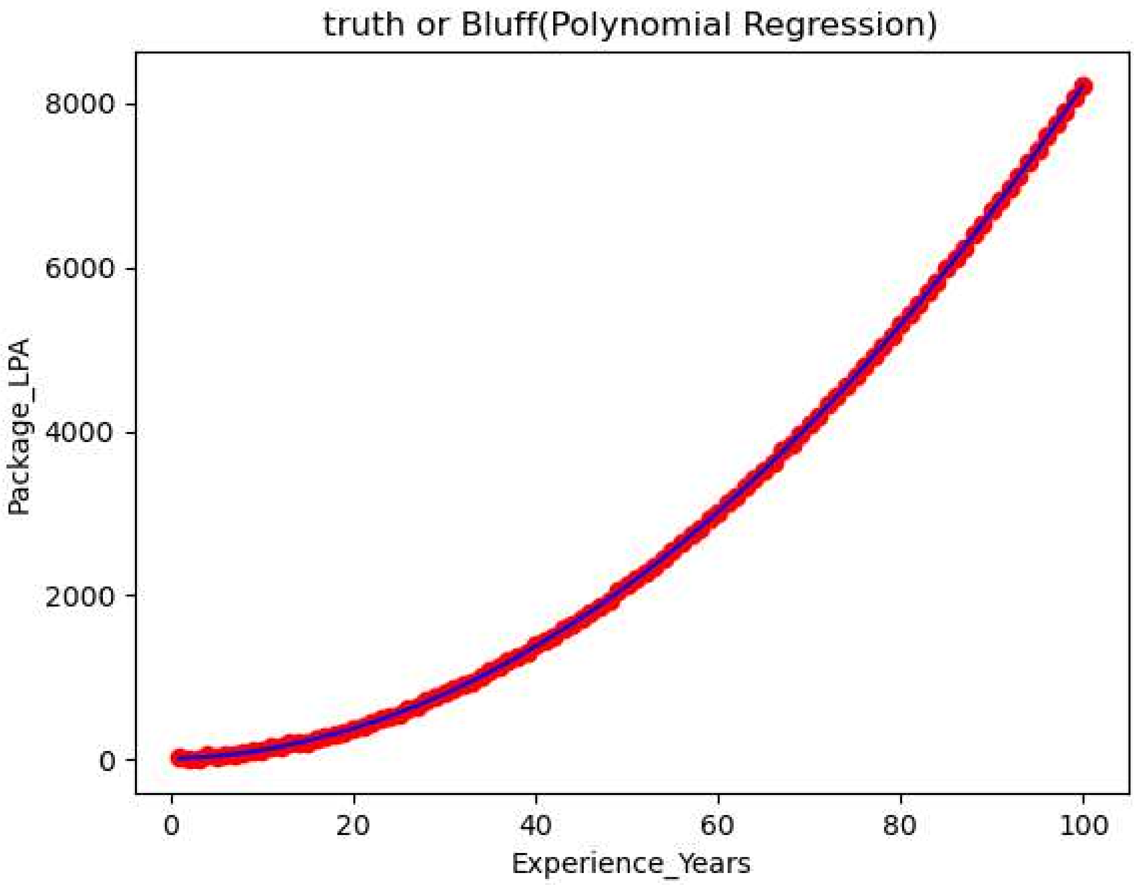
▶ Parameters

In [396...
```
plt.scatter(x,y,color='red')
plt.plot(x,lin_reg_2.predict(poly_reg.fit_transform(x)),color='blue')
plt.title('truth or bluff(polynomial regression)')
plt.xlabel('Experience_Years')
plt.ylabel('Package_LPA')
plt.show()
```



In [397...
```
x_grid = np.arange(x.values.min(), x.values.max(), 0.1)
x_grid = x_grid.reshape(-1, 1)
```

```
plt.scatter(x,y,color='red')
plt.plot(x_grid,lin_reg_2.predict(poly_reg.fit_transform(x_grid)),color='blue')
plt.title('truth or Bluff(Polynomial Regression)')
plt.xlabel('Experience_Years')
plt.ylabel('Package_LPA')
plt.show()
```



In [398…

```
lin_reg_2.predict(poly_reg.transform(np.array([[6.5]])))
```

Out[398…

```
array([[63.12300475]])
```

In [399…

```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np


r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)


print("R² Score :", r2)
print("MAE      :", mae)
print("MSE      :", mse)
```

```
R² Score : 0.9361157812203754
MAE      : 452.0923675700809
MSE      : 316064.1510463959
```

```
In [ ]:
```